

Assignment: Shape Calculator Using Object-Oriented Programming

Course

Object-Oriented Programming (Python)

Assignment Title

Design and Develop a Shape Calculator Using Encapsulation, Inheritance, and Polymorphism

Objective

The objective of this assignment is to design and implement a Shape Calculator application that demonstrates the following Object-Oriented Programming concepts:

- Encapsulation
- Inheritance
- Polymorphism

Students will create classes representing various geometric shapes and implement methods to calculate area, perimeter, and volume.

Problem Statement

A geometry software company requires a reusable software component for performing calculations on different geometric shapes. The system should be designed using Object-Oriented Programming principles to promote code reusability, maintainability, and extensibility.

Develop a class hierarchy representing various two-dimensional (2D) and three-dimensional (3D) shapes and implement methods to perform the required calculations.

Shapes to be Implemented

Two-Dimensional (2D) Shapes

1. Circle
2. Rectangle
3. Square
4. Triangle

Three-Dimensional (3D) Shapes

5. Cone
 6. Cylinder
 7. Sphere
 8. Cuboid
 9. Cube
-

OOP Requirements

1. Encapsulation

All dimensions of shapes must be stored as private data members.

Example

```
class Circle:
    def __init__(self, radius):
        self.__radius = radius
```

Access to dimensions should be provided through getter/setter methods or properties.

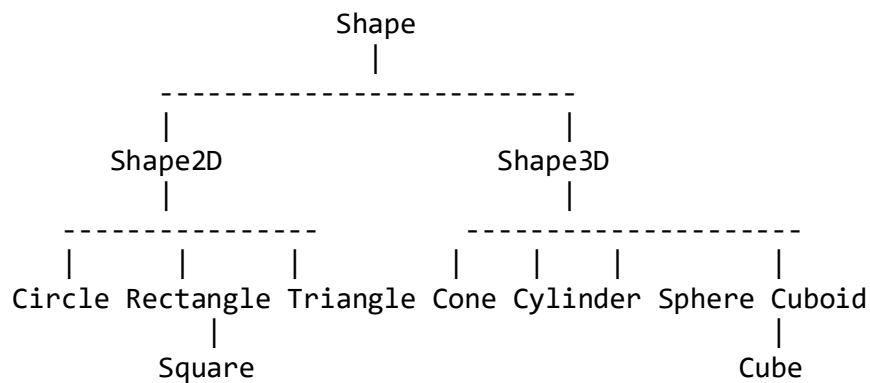
Demonstration Requirements

- Private attributes for dimensions.
 - Controlled access through methods.
 - Validation of dimensions (optional).
-

2. Inheritance

Create a class hierarchy to avoid code duplication and improve maintainability.

Suggested Hierarchy



Base Class: Shape

Attributes

- shape_name

Methods

- display()

Shape2D Class

Methods

- calculate_area()
- calculate_perimeter()

Shape3D Class

Methods

- calculate_volume()

3. Polymorphism

Different objects should respond differently to the same method call.

Example

```
shapes = [  
    Circle(5),  
    Rectangle(10, 4),  
    Square(6),  
    Triangle(4, 5, 3, 4, 5)  
]  
  
for shape in shapes:  
    print(shape.calculate_area())
```

Although the same method (`calculate_area()`) is invoked, each object should compute the result according to its own implementation.

Similarly,

```
solids = [  
    Cone(3, 5),  
    Cylinder(3, 5),  
    Sphere(4),  
    Cube(6)  
]  
  
for solid in solids:  
    print(solid.calculate_volume())
```

Each object should calculate volume differently.

Formula Requirements

Circle

$$\text{Area} = \pi r^2$$

$$\text{Circumference} = 2\pi r$$

Rectangle

$$\text{Area} = \text{Length} \times \text{Width}$$

$$\text{Perimeter} = 2(\text{Length} + \text{Width})$$

Square

$$\text{Area} = \text{Side}^2$$

$$\text{Perimeter} = 4 \times \text{Side}$$

Triangle

$$\text{Area} = \frac{1}{2} \times \text{Base} \times \text{Height}$$

$$\text{Perimeter} = \text{Side1} + \text{Side2} + \text{Side3}$$

Cone

$$\text{Volume} = (1/3)\pi r^2 h$$

Cylinder

$$\text{Volume} = \pi r^2 h$$

Sphere

$$\text{Volume} = (4/3)\pi r^3$$

Cuboid

$$\text{Volume} = \text{Length} \times \text{Width} \times \text{Height}$$

Cube

$$\text{Volume} = \text{Side}^3$$

Program Requirements

The program should:

1. Create objects for all specified shapes.
2. Store dimensions using encapsulation.
3. Demonstrate inheritance through the class hierarchy.
4. Demonstrate polymorphism by invoking common methods on collections of objects.
5. Display calculated area, perimeter, and volume values as applicable.
6. Include appropriate comments and documentation.